

Standard Operating Procedure (SOP)

for User Onboarding and Job Execution on CSIS Cluster

Purpose: -

This SOP provides users with a definitive guide to accessing the newly configured CSIS Cluster, understanding its resources, software, and policies, and successfully submitting computational jobs. This ensures consistent, secure, and efficient use of shared resources.

Scope: -

This procedure applies to all institute/department stakeholders who have been granted access to the aforementioned cluster.

Prerequisites: -

- An approved user account with valid credentials.
- An SSH client (For Example: - ssh on Linux/macOS, [PuTTY](#)/[MobaXterm](#) on Windows).
- Basic familiarity with the Linux CLI mode (Command Line Interface).

Definitions: -

Login Node/Master Node:

The entry point for editing files, compiling code, and submitting jobs.

Slurm:

The job scheduler (Simple Linux Utility for Resource Management) manages all compute resources. Slurm schedules the job and sets priorities for it based on the cluster factors.

Batch Job:

A computational task is submitted to a queue (partition) to run without user interaction.

QoS (Quality of Service):

A set of limits that override or supplement partition limits, often used to grant different priorities or capabilities (e.g., longer runtimes).

Compute Node:

Worker nodes where jobs are executed. Access is allocated by the scheduler.

Environment Modules:

Software that allows users to dynamically load pre-installed applications and their dependencies into their shell environment.

Partition:

A queue of nodes with specific resource limits (max time, cores/GPU).

SBATCH Script:

A shell script containing directives for Slurm (`SBATCH` lines) and commands to run your software.

Technical Specifications of Compute Node:

OEM & Model	DELL PowerEdge R7525
CPU:	2× AMD EPYC 7742 (128 cores total per node)
RAM:	256 GB DDR4 memory per node
OS:	Rocky Linux 8.10
Kernel:	4.18.0-553.el8_10.x86_64
NVIDIA Driver:	570.158.01
GPU:	2× NVIDIA A100 80GB GPUs per node
CUDA Cores:	6,912 per GPU
Tensor Cores:	432 per GPU
Total VRAM & CUDA Cores per Node:	160 GB VRAM; 13,824 CUDA cores
Workload Manager	Slurm 23.11.11 (https://slurm.schedmd.com/)

Screenshots of CPUs available in each node: -

CPUs

Status	Name	Processor Brand	Processor Version	Current Speed	State	Core Count
— 	CPU1 Status	AMD EPYC 7502 32-Core Processor	Model 49 Stepping 0	2500 MT/s	Presence Detected	32

Capabilities Information for Processor

Attributes	Capable	Enabled
AMD-V	Yes	Yes
No Execute	Yes	Yes

Cache Information for Processor

Attributes	Cache1	Cache2	Cache3
Level	L1	L2	L3
Maximum Size	2 MB	16 MB	128 MB
Installed Size	2 MB	16 MB	128 MB
Type	Unified	Unified	Unified
Location	Internal	Internal	Internal
Write Policy	Write Back	Write Back	Write Back
Associativity	8-Way Set-Associative	8-Way Set-Associative	16-Way Set-Associative
Error Correction Type	Parity	Parity	Multiple-bit ECC

CPUs

Status	Name	Processor Brand	Processor Version	Current Speed	State	Core Count
— 	CPU2 Status	AMD EPYC 7502 32-Core Processor	Model 49 Stepping 0	2500 MT/s	Presence Detected	32

Capabilities Information for Processor

Attributes	Capable	Enabled
AMD-V	Yes	Yes
No Execute	Yes	Yes

Cache Information for Processor

Attributes	Cache1	Cache2	Cache3
Level	L1	L2	L3
Maximum Size	2 MB	16 MB	128 MB
Installed Size	2 MB	16 MB	128 MB
Type	Unified	Unified	Unified
Location	Internal	Internal	Internal
Write Policy	Write Back	Write Back	Write Back
Associativity	8-Way Set-Associative	8-Way Set-Associative	16-Way Set-Associative
Error Correction Type	Parity	Parity	Multiple-bit ECC

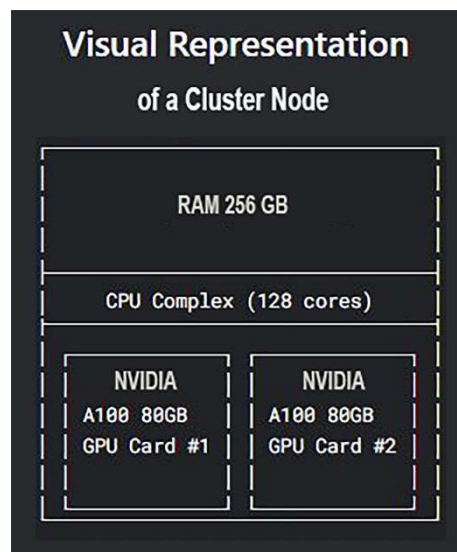
To View GPU Specifications in each node: -

- **nvidia-smi** # To see GPU utilization, including memory

NVIDIA-SMI 545.23.08				Driver Version: 545.23.08				CUDA Version: 12.3			
GPU	Name			Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC		
Fan	Temp	Perf		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.		
0	NVIDIA	A100	80GB PCIe	On	00000000:21:00.0	Off				0	
N/A	30C	P0		42W / 300W	18MiB / 81920MiB		0%	Default	Disabled		
1	NVIDIA	A100	80GB PCIe	On	00000000:81:00.0	Off				0	
N/A	29C	P0		42W / 300W	18MiB / 81920MiB		0%	Default	Disabled		
Processes:											
GPU	GI	CI		PID	Type	Process name				GPU Memory	
	ID	ID								Usage	
0	N/A	N/A		3198	G	/usr/lib/xorg/Xorg				4MiB	
1	N/A	N/A		3198	G	/usr/lib/xorg/Xorg				4MiB	

- **nvidia-smi -q | grep -E "(Product Name|FB Memory|Count)"** # To see detailed GPU properties

```
[root@csis ~]# nvidia-smi -q | grep -E "(Product Name|FB Memory|Count)"
Product Name           : NVIDIA A100 80GB PCIe
FB Memory Usage        :
Product Name           : NVIDIA A100 80GB PCIe
FB Memory Usage        :
```



DISCLAIMER

The information contained in this document is subject to change without notice. The department shall not be liable for errors contained herein or for incidental or consequential damages in connection with the performance or use of this manual.

Procedure: -

Step 1: Secure Connection (Login) to the Cluster

- I. Open a terminal window (or SSH client) on the end-user device.
- II. Connect using the following command:
ssh <username>@172.24.16.132
- III. Enter user password and any 2FA token when prompted.

```
ThinkCentre-neo-50s-Gen-3:~$ ssh rs2@172.24.16.132
The authenticity of host '172.24.16.132 (172.24.16.132)' can't be established.
ED25519 key fingerprint is SHA256:r4YlDhIWyyTAU6+Grh8cS/TFeKpZ3UuN+5wFWpcjwkg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.24.16.132' (ED25519) to the list of known hosts.
rs2@172.24.16.132's password:
Web console: https://csis.mn1.cluster_csis1:9090/ or https://172.24.16.132:9090/

Last login: Thu Aug 28 16:29:01 2025
[rs2@csis ~]$
```

Step 2: Understand the Environment

Initially, users will be on a master node as they successfully log in. User is supposed to execute the job by preparing and using Slurm files, not directly on the node's terminal console. Each user has a **300 GB** quota limit for their home directory. It may be extended as the cluster is getting more disk availability.

- ❖ To check the home directory usage details
du -sh \$HOME
du -sh /nfs_home/users/username
- ❖ To check the allotted home directory size details
quota -u \$USER
quota -u /nfs_home/users/username
- ❖ The cluster utilizes a shared, common **scratch** directory with a total capacity of **1 TB** available to all users. To simplify access, users have a **symbolic link** placed directly under their home directory that points to their dedicated subdirectory within the common scratch space.
(**cd ~/scratch** #To get into respective scratch directory)
- ❖ To prevent any single user from monopolizing the limited storage, any data residing in a user's scratch subdirectory that is **older than 30 days** will be **automatically deleted**.

```
[rs5@csis ~]$ pwd
/nfs_home/users/rs5 # Actual Path of User's home directory
[rs5@csis ~]$ cd scratch/
[rs5@csis scratch]$
scratch -> /scratch/users/rs5 # Symbolic Link to scratch directory
[rs5@csis scratch]$ ls -l
total 0
-rw-rw-r-- 1 rs5 rs5 0 Oct 2 18:59 test11
[rs5@csis scratch]$ pwd
/nfs_home/users/rs5/scratch
[rs5@csis scratch]$
[rs5@csis scratch]$ cd /scratch/users/rs5
[rs5@csis rs5]$ ls -l
total 0
-rw-rw-r-- 1 rs5 rs5 0 Oct 2 18:59 test11
[rs5@csis rs5]$ pwd
/scratch/users/rs5 # Actual Path of User's scratch directory
[rs5@csis rs5]$
```

```
[rs5@csis ~]$ quota -u $USER
Disk quotas for user rs5 (uid 1009):
Filesystem blocks quota limit grace files quota limit grace
/dev/sdb1 19892 313524224 314572800 378 0 0
```

From the above snippet, it is very clear that the cited user has just used 20MB (with 378 files), though the quota's **Soft Limit** (warning threshold) is 299GB, and the **Hard Limit** is **300GB**.

NB: - If the allotted user quota is near or exceeded with stored data, the user must archive or delete user data to ensure adequate space before new jobs can run or data can be saved.

Step 3: To transfer a File, the user can use SCP to transfer files between the local machine and the cluster.

a) To upload files from the local/user-end machine to the cluster:

- `scp /<local_directory_path>/<local_file(s)> <username>@172.24.16.132:/$HOME/<remote_directory_path>/`

b) To download files from the cluster to the local/user-end machine:

- `scp <username>@172.24.16.132:/$HOME/<remote_directory_path>/<remote_file(s)> /<local_directory_path>/`

```
fduser@root:~$ To Upload
fduser@root:~$ scp /home/fduser/test.rar rs5@172.24.16.132:/nfs_home/users/rs5
The authenticity of host '172.24.16.132 (172.24.16.132)' can't be established.
ED25519 key fingerprint is SHA256:r4YlDhIWyytAU6+Grh8cS/TFeKpZ3UuN+5wFWpcjwkg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.24.16.132' (ED25519) to the list of known hosts.
rs5@172.24.16.132's password:
test.rar                                100% 539KB 95.7MB/s 00:00
fduser@root:~$ To Download
fduser@root:~$ scp rs5@172.24.16.132:/nfs_home/users/rs5/slurm-175.out /home/fduser
rs5@172.24.16.132's password:
slurm-175.out                           100% 15 14.5KB/s 00:00
```

*The user may use any of the available/familiar GUI applications ([WinSCP](#), [FileZilla](#)) for meeting the file transferring requirements.

Step 4: Explore Available Software Modules and Resources

A key software stack is pre-installed and managed via Environment Modules. This simplifies loading complex software and its dependencies.

Key Available Modules Include:

```
[rs5@csis ~]$ module avail

----- /nfs_home/software/modulefiles/apps -----
cuda/12.8    gamess/00    jupyter/lab    miniconda/default    nvhpc/24.1

----- /nfs_home/software/modulefiles/envs -----
pytorch/1.0    tensorflow/1.0

----- /nfs_home/software/modulefiles/lang -----
python/3.9    python/3.13.5 (D)

----- /nfs_home/software/modulefiles/mpi -----
default    mpich-x86_64    mpich    openmpi-x86_64    openmpi    pmix-x86_64

----- /nfs_home/software/modulefiles/libraries -----
boost/1.84.0    fftw/3.3.10-double    hdf5/1.14.3-serial    openblas/0.3.27

----- /nfs_home/software/modulefiles -----
apps/cuda/12.8    lang/python/3.9    libs/scalapack/2.2.1
apps/gamess/00    lang/python/3.13.5 (D)    mpi/default
apps/jupyter/lab    libraries/boost/1.84.0    mpi/mpich-x86_64
apps/miniconda/default    libraries/fftw/3.3.10-double    mpi/mpich
apps/nvhpc/24.1    libraries/hdf5/1.14.3-serial    mpi/openmpi-x86_64
envs/pytorch/1.0    libraries/openblas/0.3.27    mpi/openmpi
envs/tensorflow/1.0    libs/lapack/3.12.0    mpi/pmix-x86_64

Where:
D: Default Module

If the avail list is too long consider trying:

"module --default avail" or "ml -d av" to just list the default modules.
"module overview" or "ml ov" to display the number of modules for each name.

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".
```

Short Notes about available Modules

1) lang/ (Programming Languages)

- **python/3.9**: The Python 3.9 interpreter and core libraries. Useful for projects that require compatibility with this specific, older version.
- **python/3.13.5 (D)**: The default Python 3.13.5 interpreter. This is the latest version provided by the system and is the one that will load if you just type ``module load lang/python``.

2) envs/ (Pre-configured Environments)

- **pytorch/1.0**: A pre-installed environment for the PyTorch 1.0 deep learning framework. Loading this module gives you immediate access to this specific version of PyTorch and its dependencies without needing to install it yourself.
- **tensorflow/1.0**: A pre-installed environment for the TensorFlow 1.0 deep learning framework. Useful for running older code designed for the TF1 API, which is significantly different from modern TF2.

3) mpi/ (Message Passing Interface Libraries)

- **openmpi-x86_64 (D)**: The default version of the OpenMPI library, compiled for x86_64 architecture. This is the most common high-performance MPI implementation for distributed computing and parallel jobs.
- **openmpi**: Likely a symbolic link or alternative version of the OpenMPI library.
- **mpich-x86_64**: The MPICH implementation, another high-performance and widely portable MPI library, compiled for x86_64.
- **mpich**: Likely a symbolic link or alternative version of the MPICH library.
- **pmix-x86_64**: The Process Management Interface (PMIx) library, often used as a runtime for MPI implementations and other tools in resource-managed environments (like Slurm).
- **default**: A fallback module that likely points to the system's default MPI implementation (probably ``openmpi-x86_64``).

4) libraries/ (Scientific & Numerical Libraries)

- **openblas/0.3.27**: The OpenBLAS library, an optimized implementation of the BLAS (Basic Linear Algebra Subprograms) API. Dramatically accelerates numerical and linear algebra operations in code like NumPy, R, and many scientific applications.
- **LAPACK**: Built upon BLAS, this library provides higher-level routines for solving systems of simultaneous linear equations, least-squares solutions of linear systems of equations, eigenvalue problems, and singular value problems, forming the core of many quantum chemistry algorithms.
- **ScaLAPACK** (Scalable LAPACK): A parallel extension of LAPACK for distributed-memory MIMD parallel computers. Building this library was critical for enabling the parallel execution of linear algebra operations across multiple compute nodes, which is necessary for studying large molecular systems with applications like GAMESS.

- **fftw/3.3.10-double**: The FFTW (Fastest Fourier Transform in the West) library, version 3.3.10, compiled for double-precision calculations. The standard library for high-performance Fourier transforms.
- **hdf5/1.14.3-serial**: The HDF5 library, version 1.14.3, compiled for serial (non-parallel) use. Used for managing complex data structures and storing large amounts of numerical data.
- **boost/1.84.0**: The Boost C++ source libraries, version 1.84.0. A large collection of peer-reviewed, portable C++ libraries that provide support for tasks like linear algebra, multithreading, and graph algorithms.

5) apps/ (Applications & Tools)

- **miniconda/default**: Provides the `conda` package manager and a minimal base Python environment. Essential for creating and managing your own isolated software environments for Python or R, allowing you to install specific package versions without requiring system administrator privileges.
- **jupyter/lab**: For launching JupyterLab sessions. This module is typically used with a job scheduler (like Slurm) to start an interactive session on a compute node, providing a web-based development environment for code, data, and documentation.
- **GAMESS** (General Atomic and Molecular Electronic Structure System)
It is a quantum chemistry software package. This is a comprehensive computational chemistry program for ab initio molecular orbital calculations, including methods such as Hartree-Fock, DFT, MP2, CCSD(T), and many other advanced quantum mechanical methods. Essential for researchers in computational chemistry, materials science, and molecular modeling to perform electronic structure calculations, geometry optimizations, frequency calculations, and molecular property predictions.
- **CUDA 12.8 Toolkit**
NVIDIA's parallel computing platform and programming model. Provides the compiler (nvcc), libraries, and development tools for GPU-accelerated computing. Essential for developing and running applications that leverage NVIDIA GPUs for general-purpose processing (GPGPU), including machine learning, scientific computing, and high-performance computing applications requiring massive parallelism.
- **NVIDIA HPC SDK 24.1 (nvc++, nvc, nvfortran)**
Complete suite of high-performance compilers and libraries for HPC applications. Includes the NVIDIA C++ (nvc++), C (nvc), and Fortran (nvfortran) compilers with advanced optimization features for CPU and GPU targets. Provides accelerated math libraries, MPI implementations, and development tools tailored for scientific computing, computational fluid dynamics, climate modeling, and other demanding HPC workloads requiring maximum performance across NVIDIA GPU and CPU architectures.

Step 5: How to View and Load Available Software Modules:

- `module avail` # To view all available modules
- `module avail python` # To search for specific modules (e.g., python)
- `module show lang/python/3.13.5` # To get information about a specific module
- `module load miniconda/default` # To load the default **Miniconda** module
*Mention the required module name explicitly for hassle-free setup.
- `echo $LD_LIBRARY_PATH` # To check the module library paths currently set in the same
- `module load jupyter/lab` # To load JupyterLab for interactive work
- `module list` # To see currently loaded modules
- `module unload jupyter/lab` # To unload a specified module
- `module purge` # To unload all modules

Step 6: How to View Cluster's Workload Manager Specifications & Partitions:

- `sinfo` # Overview of all partitions and node states
- `sinfo -o "%20P %5D %14F %8z %10m %10c %10G"` # Detailed view: Partition, Nodes, State, Memory, CPUs, GPUs
- `sinfo -o "%20N %10G"` # To see the physical GPU count per node

```
[rs5@csis ~]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug      up       30:00      5    idle csis.cn[1-4],csis.mn1
cpu-short* up       12:00:00   5    idle csis.cn[1-4],csis.mn1
cpu-long   up       1-00:00:00 5    idle csis.cn[1-4],csis.mn1
gpu-short  up        8:00:00   5    idle csis.cn[1-4],csis.mn1
gpu-long   up       12:00:00   5    idle csis.cn[1-4],csis.mn1
gpu-1day   up       1-00:00:00 5    idle csis.cn[1-4],csis.mn1
gpu-3day   up       3-00:00:00 5    idle csis.cn[1-4],csis.mn1
bigmem     up       12:00:00   5    idle csis.cn[1-4],csis.mn1
```

```
[rs5@csis ~]$ sinfo -o "%20P %5D %14F %8z %10m %10c %10G"
PARTITION  NODES  NODES(A/I/O/T) S:C:T  MEMORY  CPUS  GRES
debug      5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
cpu-short* 5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
cpu-long   5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
gpu-short  5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
gpu-long   5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
gpu-1day   5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
gpu-3day   5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
bigmem     5      0/5/0/5        2:32:2 240000  128  gpu:a100-8
```

```
[rs5@csis ~]$ sinfo -o "%20N %10G"
NODELIST  GRES
csis.cn[1-4],csis.mn gpu:a100-8
```


Step 7: Know the Cluster resources and availability with respect to *Partitions/QoS*.

QoS (Quality of Service) and Partition Mapping

The user may refer to the table below to know the most effective preset of Slurm for the proposed job. They must set appropriately with the most suitable *Partitions* and *QoS* for the proposed job.

QoS/Partition Name	Priority	Best For	Max Time	Max CPUs	Max (RAM) Memory	Max GPUs	Maximum Concurrent Jobs per QoS/Partition
debug	1000	Quick testing, debugging	30 min	16	48 GB	0 GPU	10
cpu-short	600	Standard CPU Jobs	12 hours	64	96 GB	0 GPU	30
cpu-long	300	Long-running CPU jobs	24 hours	32	64 GB	2 GPUs	15
gpu-short	500	GPU-accelerated jobs	8 hours	24	96 GB	GPU	10
gpu-long	350	Long-running GPU jobs	12 hours	16	80 GB	GPU	6
gpu-1day	250	GPU jobs up to 1 Day	24 hours	16	64 GB	2 GPUs	4
gpu-3day	200	GPU jobs up to 3 Days	72 hours	16	64 GB	1 GPU	2
bigmem	400	Memory-intensive jobs	8 hours	64	192 GB	0 GPU	5

Maximum Concurrent Jobs per User	Remark
5	Each individual user is limited to 5 concurrent jobs.

Group Name	Maximum Concurrent Jobs per Group	GrpTRES Limits	Remark
faculty	30	cpu=64; mem=720 GB; gres/gpu=8;	Shared limits across all faculty users to balance concurrency and resources; ie: - The cumulative resource limits that the entire faculty group can collectively consume across their jobs.

How this works:

- **MaxJobs=5** means each user can only have 5 running jobs simultaneously
- The 6th job will automatically be queued (**PD state**) with reason "**JobHeldUser**"
- As soon as one of the 5 running jobs finishes, the next queued job will start
- This applies across all partitions/QoS for that user
- Concurrent running jobs limit per faculty group also works in a similar way.

- To ensure fair resource allocation and prevent system monopolization, the cluster workload manager has been configured with per-user and overall group concurrent job limits, along with Partition/QoS resource utilization restrictions.
- This ensures all researchers get equitable access to our limited computing resources while maintaining system stability and performance for everyone.
- *Partitions/QoS Resources and Executable Job limits may be adjusted appropriately by analyzing the cluster usage on a pro-rata basis.*

Illustrative Priority Scenarios of Partition/QoS

1. <i>debug</i> (Weight -1000)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs immediately	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 15 minutes	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: Runs within 30 minutes
2. <i>cpu-short</i> (Weight - 600)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs within 30 minutes	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 1-2 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 3-4 hours during high demand
3. <i>cpu-long</i> (Weight - 300)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs within 4-6 hours	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 8-12 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 24+ hours during busy periods
4. <i>gpu-short</i> (Weight - 500)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs immediately to 30 minutes	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 1-2 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 3-4 hours during busy periods
5. <i>gpu-long</i> (Weight - 350)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs within 2-4 hours	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 4-8 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 12+ hours during high demand
6. <i>gpu-1day</i> (Weight - 250)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs within 4-8 hours	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 12-18 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 24+ hours during high demand
7. <i>gpu-3day</i> (Weight - 200)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs within 8-12 hours	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 24-48 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 3+ hours during high demand
8. <i>bigmem</i> (Weight - 400)		
<u>High Priority Job:</u> User: Low recent usage (high fair share) Result: Runs immediately to 30 minutes	<u>Medium Priority Job:</u> User: Moderate usage Result: Runs within 2-3 hours	<u>Lower Priority Job:</u> User: High recent usage (low fair share) Result: May wait 6+ hours during peak usage

Step 8: Job Scheduling and Priority Determination.

Job Scheduling Mechanism

The cluster is configured with *Slurm's (Simple Linux Utility for Resource Management)* **backfill scheduling** policy. This policy is designed to maximize system utilization by allowing the scheduler to start lower-priority jobs, provided their execution **will not delay** the expected start time of any higher-priority pending jobs. For backfill scheduling to work effectively, setting **reasonably accurate time limits** for jobs is essential, as the expected start time of pending jobs relies on the anticipated completion time of running jobs.

Job Priority Calculation

A job's priority score determines its position in the queue, with a **higher score** leading to sooner scheduling. This priority is a dynamic value that can change over time (e.g., the longer a job waits, the higher its priority may grow). The final priority is a **weighted sum** of multiple factors enabled in the *slurm.conf* file, expressed by the formula:

```
Job_priority = site_factor
               + (100) * (age_factor)
               + (0) * (assoc_factor)
               + (1000) * (fair-share_factor)
               + (200) * (job_size_factor)
               + (500) * (partition_factor)
               + (1000) * (QoS_factor)
               + SUM (TRES_weight_cpu * TRES_factor_cpu, ...)
               - nice_factor
```

In this formula, the **factors** are normalized floating-point numbers ranging from 0.0 to 1.0, while the **weights** are unsigned integers that define the relative importance of each factor.

Factor	Weight	Description
Fair-Share Factor	1000	The most influential factor. It determines scheduling order based on the user's allocated resources versus their resource consumption history (usage).
QoS Factor	1000	Based on an integer priority assigned to the Quality of Service (QoS) requested by the job.
Partition Factor	500	Based on an integer priority assigned to the partition the job requests to run in.
Job Size Factor	200	Correlates to the number of nodes or CPUs the job has requested.
Age Factor	100	Represents the length of time a job has been waiting in the queue, providing a boost to older jobs.
Association Factor	0	Based on an integer priority assigned to the job's association (account/user), normalized to the system's highest association priority.
TRES Weights	Varies	Sum of factors based on requested Trackable RESources (TRES), such as CPU, memory, etc.
Nice Factor		It allows users to voluntarily lower their job's priority.

Step 9: Slurm Script Setup Instructions for Users:

- 1) Create a job script (my_test_job.sh) that mentions the most appealing partition name, requests appropriate resources, and loads necessary modules. Users may use vi, vim, or nano editors as per their comfort level.

vi my_test_job.sh (File extension may be .slurm, .sh or .sbatch)

- 2) Slurm File Syntax: Declaration, Explanation, and Example.

```
#!/bin/bash
#SBATCH --job-name=my_test_job      # Name shown in queue (helps identify your job)
#SBATCH --output=test_%j.out        # Output log file (%j = job ID)
#SBATCH --error=test_%j.err         # Error log file (%j = job ID)
#SBATCH --partition=short           # Use partition that matches your resource needs
#SBATCH --qos=short                 # Quality of Service level (enforces limits)
#SBATCH --ntasks=1                  # Number of separate processes/tasks (1 for single process)
#SBATCH --cpus-per-task=16          # CPUs per task (for multi-threaded applications)
#SBATCH --mem=48G                   # Total memory needed (48GB for this example)
#SBATCH --time=00:30:00             # Wall-clock time limit (HH:MM:SS)

# Optional: Load required modules
module load lang/python/3.13.5      # Replace the module name with the actually required
module load miniconda3/4.12.0

echo "Starting the test job on $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "Allocated CPUs: $SLURM_CPUS_ON_NODE"
echo "Allocated Memory: $SLURM_MEM_PER_NODE"

# Alternative: If using a conda environment, uncomment and modify the below accordingly.
# source /nfs_home/software/miniconda/etc/profile.d/conda.sh
# conda activate your_env_name
python /path/to/your/script/my_script.py # Replace with actual computation file with path
echo "Test job completed successfully at $(date)"
```

- 3) Make it executable:

chmod +x my_test_job.sh

- 4) Submit the job:

sbatch my_test_job.sh

Best Practices for Users

For a fast turnaround and to ensure fairness in resource availability for all users,

- Use appropriate QoS levels.
- Don't over-request hardware resources - it affects the fair share priority...!

```
#SBATCH --cpus-per-task=4      #Request only needed resources
#SBATCH --mem=8G
#SBATCH --time=01:00:00       # Request realistic time
```

- Complete jobs promptly.

Step 10: To monitor and know about the executed job

1) To monitor and manage the job in progress:

- `squeue -u $USER` # To check the status of the job in the cluster
- `scontrol show job <jobid>` # To know the executed job status, particularly
- `sinfo -o "%20P %10a %10c %10m %10G %60"` # To show resource utilization (CPU, memory, GPU loads)
- `sprio -u $USER` # To check job priorities
- `sshare -u $USER` # To check the fair share priority status
- `tail -f test_%j.out` # To check the output file of the job in progress
- `tail -f test_%j.err` # To check the error file of the job in progress
- `scontrol update job <jobid> set <new attribute value>` # To change attributes of the submitted job.
(`scontrol update jobid=106 set TimeLimit=03:00:00` # Example)
- `scontrol hold <jobid>` # To hold a submitted job.
- `scontrol release <jobid>` # To release a previously held job.
- `scancel <jobid>` # To cancel a pending or running job

2) To know the completed job, and to know about pending jobs:

- `sacct -u $USER` # To view your job history
- `cat test_%j.out` # To check the Slurm output file of the completed job
- `cat test_%j.err` # To check the Slurm error file of the completed job
- `squeue -t pending -o "%8i %.12P %.20j %.8u %.2t %.10M %.6D %25R"` # To see all pending jobs and their reasons
- `squeue -t pending --start` # To see estimated start times for pending jobs
- `seff <jobid>` # To get a detailed efficiency report after the job completes

Quick Reference for One-liner Execution Syntax

The Users may utilize the following job execution syntax or standard script templates based on their specific requirements.

1) Quick job submission examples:

- `sbatch --partition=debug --time=00:30:00 --mem=16G script.sh`
- `sbatch --partition=short --time=04:00:00 --gres=gpu:1 script.sh`
- `sbatch --partition=long --time=24:00:00 --mem=48G script.sh`
- `sbatch --partition=bigmem --mem=180G script.sh`

- 2) A few standard Slurm script templates are included here for reference. The syntax for loading the required module from the available modules is included for illustrative purposes.

1. Basic CPU Job with Miniconda (short partition)

```
#!/bin/bash
#SBATCH --job-name=conda_job
#SBATCH --partition=short
#SBATCH --time=04:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --output=conda_%j.out
#SBATCH --error=conda_%j.err

# Load required modules
module load miniconda3/4.12.0

# Initialize conda from NFS shared installation
source /nfs_home/software/miniconda/etc/profile.d/conda.sh

# Activate your environment
conda activate my_env

# Verify environment
echo "Conda environment: $CONDA_DEFAULT_ENV"
echo "Python path: $(which python)"

# Your code here
python my_script.py"=== Job Completed ==="
```

2. GPU Job with Miniconda (gpu-short partition)

```
#!/bin/bash
#SBATCH --job-name=conda_gpu_job
#SBATCH --partition=gpu-short
#SBATCH --gres=gpu:a100-80gb:1
#SBATCH --time=04:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --output=gpu_conda_%j.out
#SBATCH --error=gpu_conda_%j.err
echo "=== GPU Job Started ==="
echo "Job ID: $SLURM_JOB_ID"
echo "GPU allocated: $CUDA_VISIBLE_DEVICES"
# Load required modules
module load miniconda3/4.12.0
module load cuda/11.7
# Initialize conda
source /nfs_home/software/miniconda/etc/profile.d/conda.sh
# Activate environment with GPU support
conda activate pytorch_gpu
# Verify GPU access
echo "Python: $(which python)"
nvidia-smi
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}'); print(f'GPU count: {torch.cuda.device_count()}')"
# GPU-accelerated script
python train_model.py --batch-size 32 --epochs 10 --gpu-id 0

echo "=== GPU Job Completed ==="
```

3. Debug Job with Miniconda (debug partition)

```
#!/bin/bash

#SBATCH --job-name=conda_debug

#SBATCH --partition=debug

#SBATCH --time=00:30:00

#SBATCH --ntasks=1

#SBATCH --cpus-per-task=4

#SBATCH --mem=16G

#SBATCH --output=debug_%j.out

#SBATCH --error=debug_%j.err

echo "=== Debug Job ==="

echo "Quick test of conda environment"

# Source conda (adjust path as needed)

source /nfs_home/software/miniconda/etc/profile.d/conda.sh

# List available environments

echo "Available conda environments:"

conda env list

# Test basic environment

conda activate base

python -c "

import sys

print(f'Python {sys.version}')

import numpy as np

print(f'NumPy {np.__version__}')

print('Debug test completed successfully')

"

echo "=== Debug Completed ==="
```

4. Long-running CPU Job with Miniconda (long partition)

```
#!/bin/bash

#SBATCH --job-name=conda_long_job

#SBATCH --partition=long

#SBATCH --time=1-00:00:00

#SBATCH --ntasks=1

#SBATCH --cpus-per-task=12

#SBATCH --mem=40G

#SBATCH --output=long_%j.out

#SBATCH --error=long_%j.err


echo "=== Long-running Job ==="

echo "Duration: 24 hours"


# Setup conda

source /nfs_home/software/miniconda/etc/profile.d/conda.sh

conda activate research_env


# Long-running analysis

python extended_analysis.py \

    --threads $SLURM_CPUS_PER_TASK \

    --memory $((($SLURM_MEM_PER_NODE / 1024))G

echo "=== Long Job Completed ==="
```


5. Memory-Intensive Job with Miniconda (bigmem partition)

```
#!/bin/bash

#SBATCH --job-name=conda_bigmem
#SBATCH --partition=bigmem
#SBATCH --time=08:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --mem=180G
#SBATCH --output=bigmem_%j.out
#SBATCH --error=bigmem_%j.err

echo "=== Big Memory Job ==="
echo "Allocated memory: $SLURM_MEM_PER_NODE MB"

# Setup conda for memory-intensive work
source /nfs_home/software/miniconda/etc/profile.d/conda.sh
conda activate bigdata_env

# Memory-intensive processing
python process_large_data.py \
    --workers $SLURM_CPUS_PER_TASK \
    --memory $(( $SLURM_MEM_PER_NODE / 1024 ))G

echo "=== Big Memory Job Completed ==="
```

6. Multi-GPU Job with Miniconda (gpu-short partition)

```
#!/bin/bash
#SBATCH --job-name=conda_multi_gpu
#SBATCH --partition=gpu-short
#SBATCH --gres=gpu:a100-80gb:2
#SBATCH --time=04:00:00
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --output=multi_gpu_%j.out
#SBATCH --error=multi_gpu_%j.err
echo "=== Multi-GPU Job ==="
echo "GPUs allocated: $CUDA_VISIBLE_DEVICES"
# Setup conda with GPU support
source /nfs_home/software/miniconda/etc/profile.d/conda.sh
conda activate multi_gpu_env
# Verify multi-GPU setup
python -c "
import torch
print(f'Available GPUs: {torch.cuda.device_count()}')
for i in range(torch.cuda.device_count()):
    print(f'GPU {i}: {torch.cuda.get_device_name(i)} -
{torch.cuda.get_device_properties(i).total_memory/10243:.1f}GB')
"
# Multi-GPU training
python -m torch.distributed.launch --nproc_per_node=2 \
    train_multi_gpu.py --batch-size 64 --epochs 20

echo "=== Multi-GPU Job Completed ==="
```

“Users may follow these given standard Slurm script templates and modify the required hardware resources, module, environment name, main inputting computation script, and Partition/QoS/other critical parameters as needed for their specific workloads.”

To Integrate Slurm with Major Development Platforms

1) VS Code (with Miniconda)

- **Source Conda shell** `Source /nfs_home/software/miniconda/etc/profile.d/conda.sh`
- **Activate environment** `conda activate my_env`
- **Run script** `python my_script.py`
- **Deactivate** `conda deactivate`

*Use **Slurm Dashboard** extension in VS Code to submit/monitor jobs.

2) JupyterLab

- **Load Jupyter module** `module load jupyter/lab`
- **Start JupyterLab on the cluster** `mkdir -p ~/.jupyter/runtime`
`export JUPYTER_RUNTIME_DIR=~/.jupyter/runtime`
`jupyter lab --no-browser --port=8855 --ip=0.0.0.0`
- **Port forwarding (from local PC)** `ssh -L 8855:localhost:8855 username@<cluster_IP>`
- **Access in browser** <http://localhost:8855/lab?token=<ID>>

*Use **JupyterLab-Slurm** or **Slurm-Magic** for job submission/monitoring.

*Check or assign a free port (example: 8855)

`netstat -tlnp | grep 8855`

*If the port is already in use, choose another available port.

3) Conda Environment for Jupyter

- **Install kernel** `conda install ipykernel`
`python -m ipykernel install --user --name my_env --display-name "My Conda Environment"`

*Now the environment will appear as a selectable kernel in Jupyter.

Users are hereby encouraged to make use of authorized online resources to overcome the learning curve and become familiar with the cluster and its operations.

Step 11: Getting Help and Support:

- ✓ Users may email their queries or requests for additional module expansions to the support team.
- ✓ In case of any technical error, users are requested to include the corresponding job ID and the relevant section of the slurm-.out file in their email.
- ✓ Support can be reached through the following channels:

- **Email:** vincem@pilani.bits-pilani.ac.in
- **In-Person Assistance:** CSIS Lab (Room 6017).